

Exception Handling

An exception is an event that stops the normal flow of a program. Java signals it by creating an exception object and throwing it. Exception handling is how we catch that object and decide what happens next.

A reading order:

What an exception is → try and catch → finally → The hierarchy
→ Checked and unchecked → throw and throws → Multiple catch
→ Custom exception → try-with-resources

1. What an exception is

When something goes wrong, Java throws an exception. If nothing catches it, the program stops at that line.

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
  
        System.out.println("Before");  
        System.out.println(numbers[5]);  
        System.out.println("After");  
    }  
}
```

Output:

```
Before  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:  
    Index 5 out of bounds for length 3  
    at Main.main(Main.java:6)
```

`After` never printed. The program ended at the bad line.

The message tells us three things: what went wrong, the value that caused it, and the line it happened on.

2. try and catch

`try` wraps the code that might fail. `catch` says what to do when it does.

```

public class Main {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3};

        try {
            System.out.println(numbers[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        System.out.println("Program continues");
    }
}

```

Output:

```

Caught: Index 5 out of bounds for length 3
Program continues

```

The same bad line ran, but the program did not stop. That is what `catch` is for.

3. finally

`finally` runs whether or not an exception was thrown. It is where we release something we opened.

```

public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Opening the file");
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        } finally {
            System.out.println("Closing the file");
        }
    }
}

```

Output:

```

Opening the file
Caught: / by zero
Closing the file

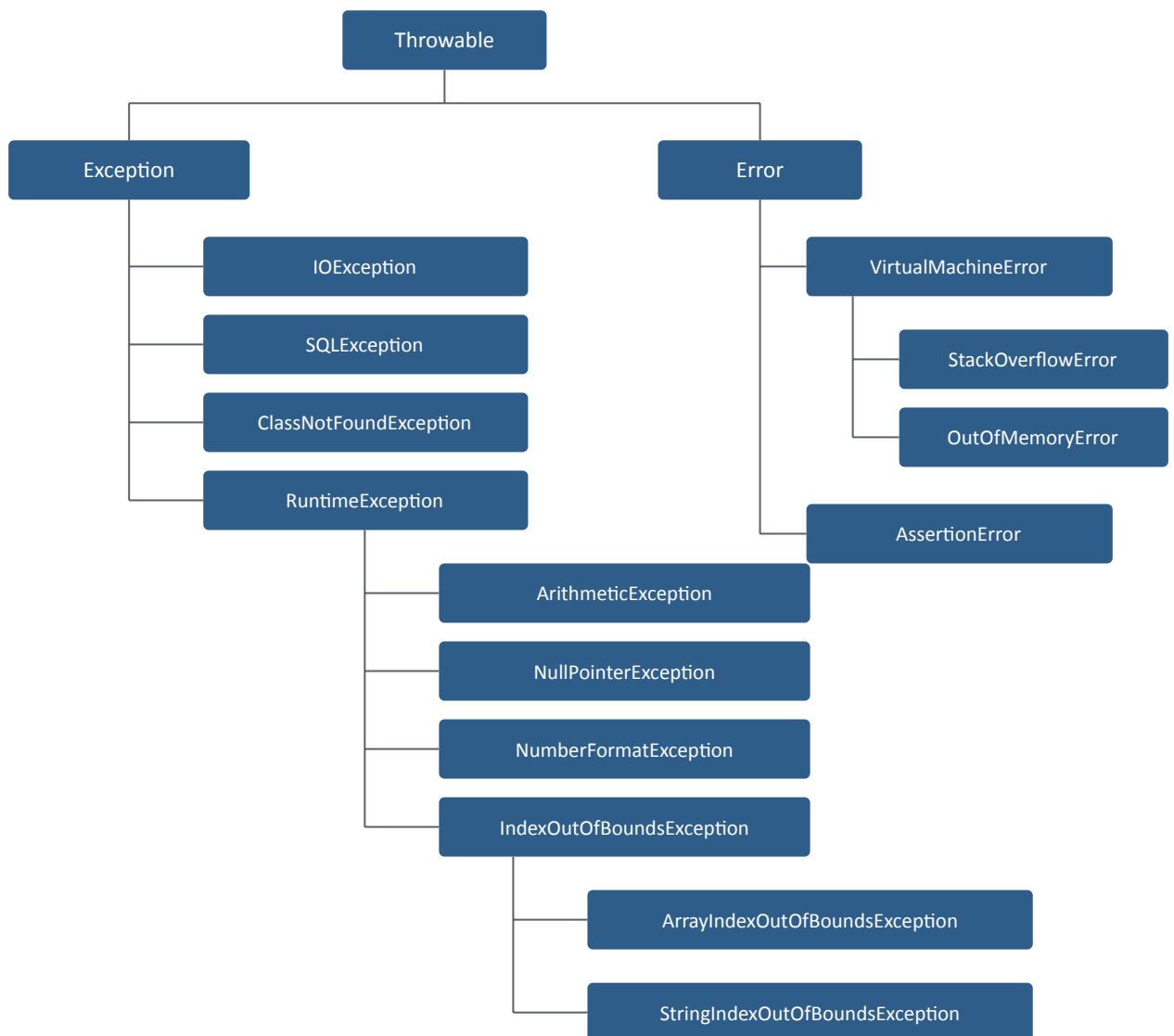
```

The line printing `result` never ran, and `finally` still did.

```
try    → code that might fail
catch  → what to do when it does
finally → what to do either way
```

4. The exception hierarchy

Everything that can be thrown comes from `Throwable`. It splits into two branches.



Everything under `RuntimeException` is unchecked. Everything else under `Exception` is checked.

Branch	Examples	Do we catch it
Error	OutOfMemoryError, StackOverflowError	No. The program cannot recover
RuntimeException	NullPointerException, IllegalArgumentException	Sometimes. Usually it is a bug to fix
Other Exception	IOException, SQLException	Yes. The compiler insists

Error means the environment failed, not our logic. Catching **OutOfMemoryError** rarely helps, because there is no memory left to run the handler.

5. Checked and unchecked exceptions

A checked exception must be handled or declared, and the compiler refuses to build until we do. An unchecked exception is anything under **RuntimeException**, and nothing is forced.

```
import java.io.FileReader;
import java.io.IOException;

public class Main {

    // IOException is checked, so the method must declare it
    static void readFile() throws IOException {
        FileReader reader = new FileReader("data.txt");
        reader.close();
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("Could not read the file");
        }

        // NullPointerException is unchecked, nothing is forced
        String text = null;
        try {
            System.out.println(text.length());
        } catch (NullPointerException e) {
            System.out.println("Text was null");
        }
    }
}
```

Output when **data.txt** does not exist:

Could not read the file
Text was null

	Checked	Unchecked
Parent	Exception	RuntimeException
Compiler	Forces handle or declare	Forces nothing
Cause	Something outside the program	Usually a bug in the code
Examples	IOException, SQLException	NullPointerException, ArithmeticException

Checked → something outside us failed, plan for it
Unchecked → we made a mistake, fix the code

6. throw and throws

`throw` raises an exception now. `throws` declares what a method may throw, so the caller knows.

```
public class Main {  
  
    static void setAge(int age) {  
        if (age < 0) {  
            throw new IllegalArgumentException("Age cannot be negative");  
        }  
        System.out.println("Age set to " + age);  
    }  
  
    public static void main(String[] args) {  
        setAge(25);  
  
        try {  
            setAge(-5);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Caught: " + e.getMessage());  
        }  
    }  
}
```

Output:

Age set to 25
Caught: Age cannot be negative

Checking the argument and throwing early is better than letting a bad value reach the rest of the program and fail somewhere confusing.

7. Catching more than one exception

One `catch` can handle several types, separated by `|`.

```
public class Main {  
  
    static void process(String value) {  
        try {  
            int number = Integer.parseInt(value);  
            System.out.println(100 / number);  
        } catch (NumberFormatException | ArithmeticException e) {  
            System.out.println(e.getClass().getSimpleName() + ": " + e.getMessage());  
        }  
    }  
  
    public static void main(String[] args) {  
        process("20");  
        process("abc");  
        process("0");  
    }  
}
```

Output:

```
5  
NumberFormatException: For input string: "abc"  
ArithmeticException: / by zero
```

When we write separate `catch` blocks instead, the order matters. A subclass must come before its parent:

```
try {  
    readFile();  
} catch (Exception e) {           // too broad, and it comes first  
} catch (IOException e) {        // Error: already caught by Exception  
}
```

The compiler rejects this, because nothing could ever reach the second block.

8. Custom exceptions

We write our own by extending `Exception` for a checked one, or `RuntimeException` for an unchecked one.

```

class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance = 1000;

    void withdraw(double amount) {
        if (amount > balance) {
            throw new InsufficientFundsException(
                "Balance is " + balance + ", cannot withdraw " + amount);
        }
        balance -= amount;
        System.out.println("Withdrew " + amount + ", balance is now " + balance);
    }
}

```

Using it:

```

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        account.withdraw(300);

        try {
            account.withdraw(5000);
        } catch (InsufficientFundsException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
}

```

Output:

```

Withdrew 300.0, balance is now 700.0
Caught: Balance is 700.0, cannot withdraw 5000.0

```

A named exception says what went wrong at the point it is caught. `InsufficientFundsException` tells us more than `IllegalArgumentException`, and far more than a `false` return value.

9. try-with-resources

Anything that implements `AutoCloseable` can be opened inside the brackets of `try`. Java closes it when the block ends, whether or not an exception was thrown.

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        try (BufferedReader reader = new BufferedReader(new FileReader("data.txt")))
        {
            System.out.println(reader.readLine());
        } catch (IOException e) {
            System.out.println("Could not read the file");
        }
    }
}
```

Output when `data.txt` does not exist:

```
Could not read the file
```

The older way needed `finally`, a null check, and a second `try` around the close:

```
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("data.txt"));
    System.out.println(reader.readLine());
} catch (IOException e) {
    System.out.println("Could not read the file");
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            // nothing useful to do here
        }
    }
}
```

Both do the same work. Prefer the first.

Practical rules

Rule	Why
Never leave a <code>catch</code> block empty	The failure disappears and nobody finds out
Catch the specific type, not <code>Exception</code>	A broad catch hides errors we did not expect
Put a useful value in the message	"Balance is 700.0" says more than "invalid amount"
Do not use exceptions for normal flow	They are slow and they hide the real path
Pass an exception on if we cannot handle it	The caller may know what to do
Use <code>try</code> -with-resources for anything we open	It closes even when something fails

An empty `catch` is the one worth repeating, because it is the easiest to write and the hardest to debug:

```
try {
    saveOrder(order);
} catch (Exception e) {
    // the order was never saved and nothing says so
}
```

The pieces together

Idea	In one line
Checked	The compiler makes us handle it, because the environment can fail
Unchecked	A bug in the caller, so it should reach a log and not a <code>catch</code>
Catch what we can act on	Anything else belongs further up
<code>finally</code> and try-with-resources	Release happens on every path, including the thrown one
Never swallow	An empty <code>catch</code> turns a failure into a wrong answer
Custom exceptions	Carry the reason, so the caller does not parse a message

For example, in an order service. Reading the payment file throws a checked `IOException`, so the method declares it. A negative order total throws an unchecked `IllegalArgumentException`, because that is a bug in the caller. A card that is declined throws our own `PaymentDeclinedException`, which carries the reason. The file is opened with `try`-with-resources, so it closes whatever happens.

Take one method from a real project and decide, for each thing that can fail, whether it is our bug or the environment failing.